

The Two-Tower Recommender

Matching the right **article** to the right **publisher**

A visual, plain-language tour of how two neural "towers" learn to predict whether a story belongs with a publisher
— scored from 0 to 1.

Article features

Publisher profile

Collaborative filtering

Match score

• THE CHALLENGE

Too many stories, too many publishers

Hand-matching content to the right outlet is slow, inconsistent, and impossible at scale.

● **Thousands of articles**

New stories arrive every hour across every topic and region.

● **Hundreds of publishers**

Each has its own focus, audience, region and appetite.

● **Manual & slow**

Editors can't review every possible pairing by hand.

● **The goal**

Auto-score every (article, publisher) pair from 0 to 1.

We want a model that, for any story and any publisher, instantly answers: “How likely is this a good match?”

Think of it as a **matchmaker**

● 1. Two profiles

The article describes itself; the publisher describes itself.

● 2. Two fingerprints

The model turns each profile into a compact list of numbers.

● 3. Compare

It checks how well the two fingerprints fit together.

● 4. A match score

Out comes a number from 0 (no) to 1 (perfect fit).

Just like a dating app compares two people, the two-tower model compares a story and a publisher — and predicts the chance they belong together.

● FIRST, A KEY CONCEPT

What is an embedding?

A way to turn words and features into coordinates on a map, where similar things sit close together.

● Meaning becomes location

"Human rights in Brazil" lands near "Politics in LATAM" — far from "Pacific Rim chips".

● A list of numbers

Each item becomes a vector, e.g. [0.21, -0.88, 0.04, ...] — its fingerprint.

● Closeness = relevance

If two fingerprints are near each other, the items are related.

An embedding tower is simply the part of the model that produces these fingerprints.

Two towers, built independently

Each tower is its own small neural network. They never share inputs — they only meet at the very top.



Because the towers are separate, we can pre-compute publisher fingerprints once and reuse them — making real-time scoring extremely fast.

● TOWER 1

The Article Tower

It takes everything we know about the story and compresses it into one blue fingerprint.

● **Category / Topic**

Politics, human rights, tech, sports...

● **Quality score**

Editorial quality, sourcing, depth.

● **Recency**

Is it breaking / timely, or evergreen?

● **Region covered**

LATAM, Pacific Rim, EMEA, global...

● **Format & length**

Brief, feature, investigation, op-ed.

● **Language & tone**

Language, sentiment, sensitivity.

These features flow up the tower — out comes the article embedding.

● TOWER 2

The Publisher Tower

A publisher is best described in words — so this tower reads a short profile and structured facts.

“Independent newsroom focused on politics and human rights, with strong coverage across LATAM. High weekly publishing capacity and a long, reliable track record with us.”

● **Topical interests**

Politics, human rights, climate...

● **Regions covered**

LATAM, Pacific Rim, EMEA...

● **Capacity**

How much they can publish.

● **History with us**

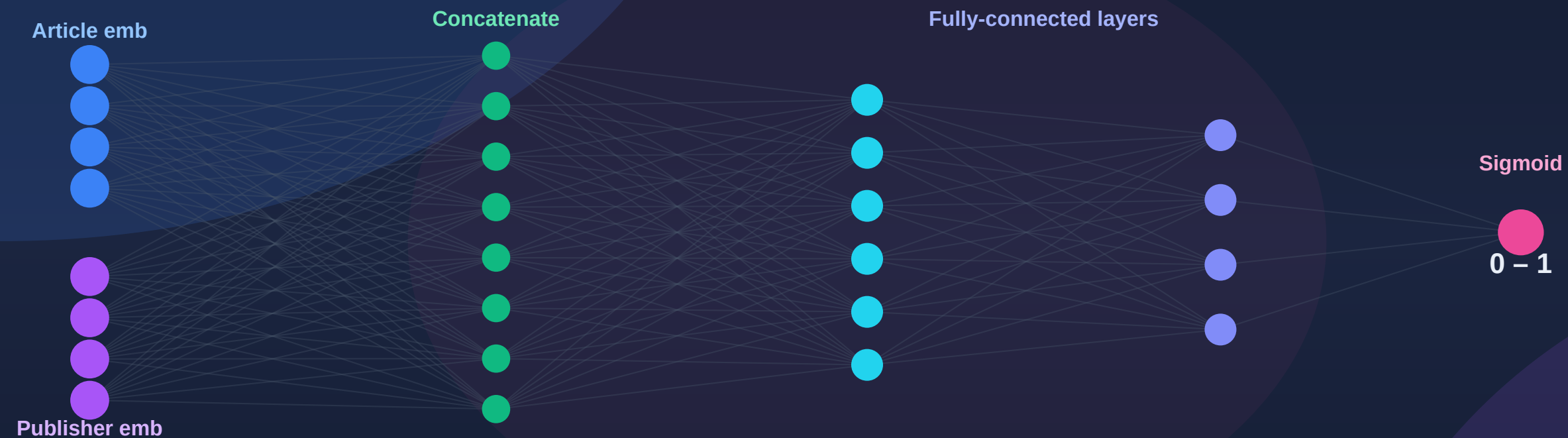
Relationship length & reliability.

A text encoder turns the paragraph into numbers, combined with the structured facts — the publisher embedding.

• THE SCORING HEAD

Where the towers **meet**

Join the two fingerprints, pass them through a fully-connected network, and squeeze the result into a 0–1 score.



Concatenate → fully-connected (dense) layers → sigmoid → a probability between 0 and 1.

Classic two-tower setups compare fingerprints with a simple dot-product for ultra-fast retrieval; adding this dense interaction head trades a little speed for sharper ranking accuracy.

Adding collaborative filtering

Content features describe what things are. Collaborative filtering learns from who interacted with what.

● ID embeddings (hybrid)

Give every publisher & article a learned ID vector, concatenated with the content fingerprint — capturing behavior the words don't.

● History pooling

Average the embeddings of articles a publisher previously accepted to build a "taste vector".

● Graph / GNN signal

Model the publisher-article interaction graph (e.g. LightGCN) and learn embeddings from the connections themselves.

● Solves cold-start

Content handles brand-new items; collaborative signal sharpens matches for familiar ones. Best of both.

The winning recipe is usually hybrid: content + collaborative in the same two towers.

Teaching the model from past matches

● Positive & negative pairs

Pairs that were accepted -> label 1. Pairs ignored or never shown (sampled) -> label 0.

● Binary cross-entropy

The model is penalised whenever its predicted score is far from the true 0/1 label.

● Trained jointly

Both towers and the scoring head learn together via gradient descent, sharpening every fingerprint.

Over millions of historical article-publisher interactions, the towers learn to place good matches close together and bad matches far apart.

• A WORKED EXAMPLE

One story, scored against three publishers

INCOMING ARTICLE

“Crackdown on press freedom across LATAM”

Politics

Human rights

LATAM

Breaking

High quality

Publisher A - LATAM Politics & Rights desk

Politics · human rights · LATAM · high capacity

0.93

Recommend

Publisher B - Global human-rights review

Human rights · global · medium capacity

0.71

Consider

Publisher C - Pacific Rim tech & business

Technology · business · Pacific Rim

0.08

Skip

The model ranks publishers instantly — editors act on a short, confident shortlist.

• WHY IT MATTERS

The business **payoff**

● **Scale & speed**

Millions of pairings scored in milliseconds — far beyond manual reach.

● **Better relevance**

Sharper matches lift acceptance and audience engagement.

● **Cold-start ready**

Content features score brand-new stories and publishers from day one.

● **Frees editors**

People focus on judgment calls, not screening every pair.

● **Self-improving**

Every new interaction makes the next recommendation better.

● **Consistent & auditable**

Data-driven scores replace ad-hoc, subjective routing.

4

Pre-computed publisher fingerprints make it fast, scalable, and self-improving in production.

The right story reaches the right publisher — automatically, and at scale.

Presentation by Omar Kamal Hosney · omkamal.github.io